



CHAPTER 08

Functions & Recursions



- A **function** is a group of statements performing a specific task.
- When a program gets bigger in size and its complexity grows, it gets difficult for a program to keep track on which piece of code is doing what!
- A function can be **reused** by the programmer in a given program any number of times.

Functions make our code organized and easy to use!



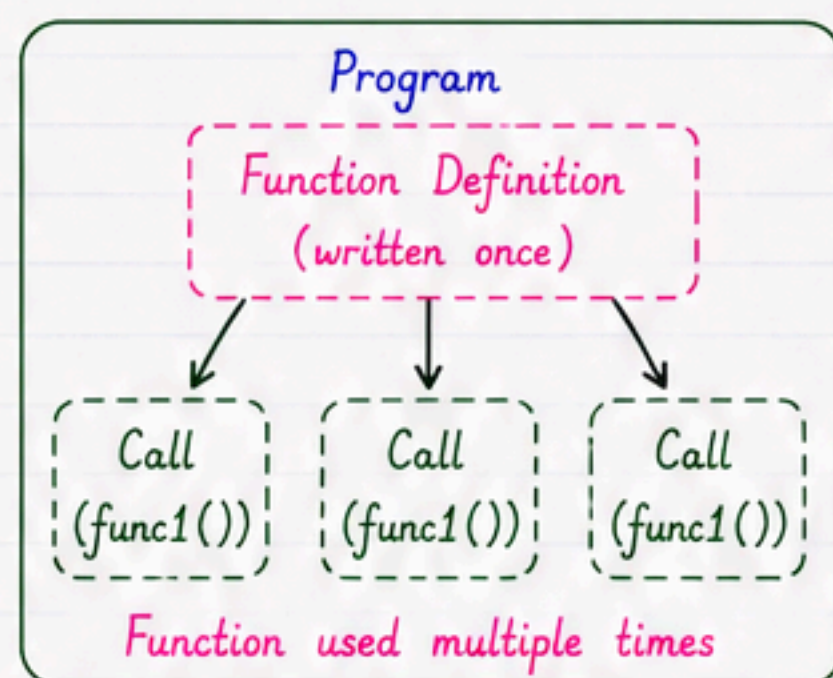
Example And Syntax Of A Function

The **syntax** of a function looks as follows:

```
def func1():  
    print('hello')
```



This function can be called any number of times, anywhere in the program.

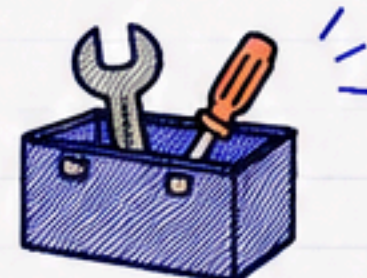


Function Call

Whenever we want to call a function, we put the name of the function followed by parentheses as follows:

```
func1() # This is called function call.
```

Calling a function is like using a ready-made tool!



Function Definition

The part containing the exact set of instructions which are executed during the function call.

```
def func1():  
    print('hello')
```

This is the function definition (written once)

Quick Quiz:

Write a program to greet a user with "Good day" using functions.

```
def greet():  
    print("Good day")  
greet() # function call
```



Good day!



Types Of Functions In Python

There are two types of functions in python:

- **Built in functions** (Already present in python)
- **User defined functions** (Defined by the user)

Examples of built in functions includes `len()`, `print()`, `range()` etc.

The `func1()` function we defined is an example of user defined function.

Examples of Built in Functions

`len()` `print()` `range()`

finds length
of items

prints
output

generates
a sequence





Functions With Arguments

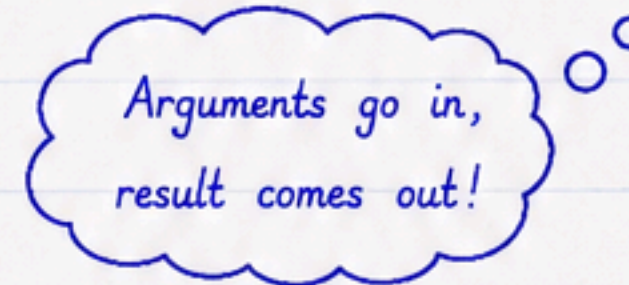
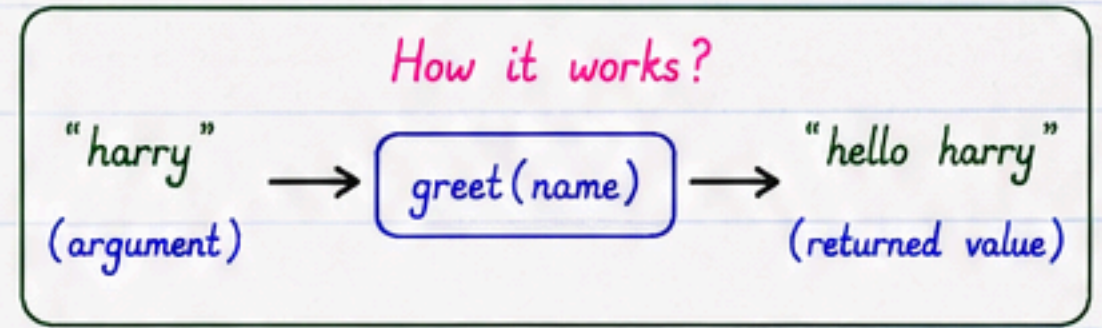


- A function can accept some value it can work with.
We can put these values in the parentheses.
- A function can also return value as shown below:

```
def greet(name):
    gr = "hello" + name
    return gr
```

```
a = greet("harry")
```

#a will now contain "hello harry"



Default Parameter Value

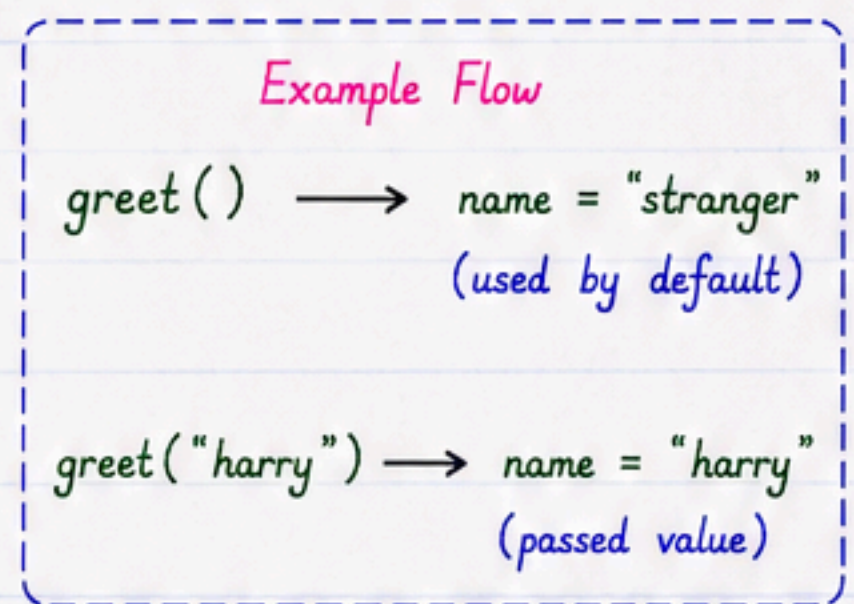
We can have a value as default as default argument in a function.
If we specify `name = "stranger"` in the line containing `def`,
this value is used when no argument is passed.

Example:

```
def greet(name = "stranger"):
    # function body
```

```
greet() # name will be "stranger" in function body (default)
```

```
greet("harry") # name will be "harry" in function body (passed)
```



Defaults
make life easy!



Recursion

- Recursion is a function which calls itself.
- It is used to directly use a mathematical formula as function.

Example:

```
factorial(n) = n × factorial(n-1)
```

This function can be defined as follows:

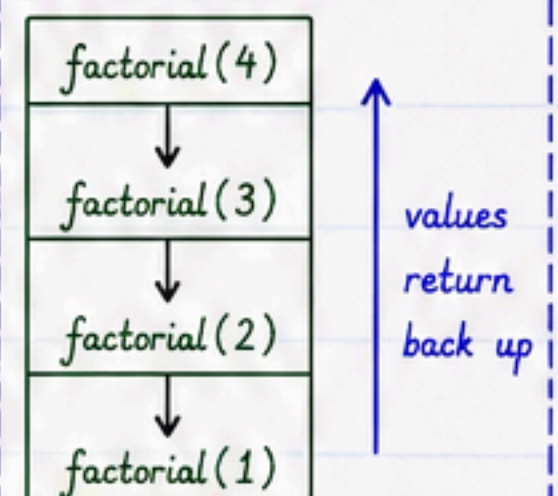
```
def factorial(n):
    if i == 0 or i == 1:
        return 1
    else:
        return n * factorial(n-1)
```

How Recursion Works?
(factorial(4))

```

    factorial(4) = 4 × factorial(3)
    factorial(3) = 3 × factorial(2)
    factorial(2) = 2 × factorial(1)
    factorial(1) = 1 (base case)
  
```

Call Stack
(Conceptual)



Functions
make code
clean & reusable!



Recursion:
A function that
believes in itself!





The factorial works as follows:

Example: Find factorial of 4

$$\text{factorial}(4) = 4 \times 3 \times 2 \times 1 = \underline{\underline{24}}$$

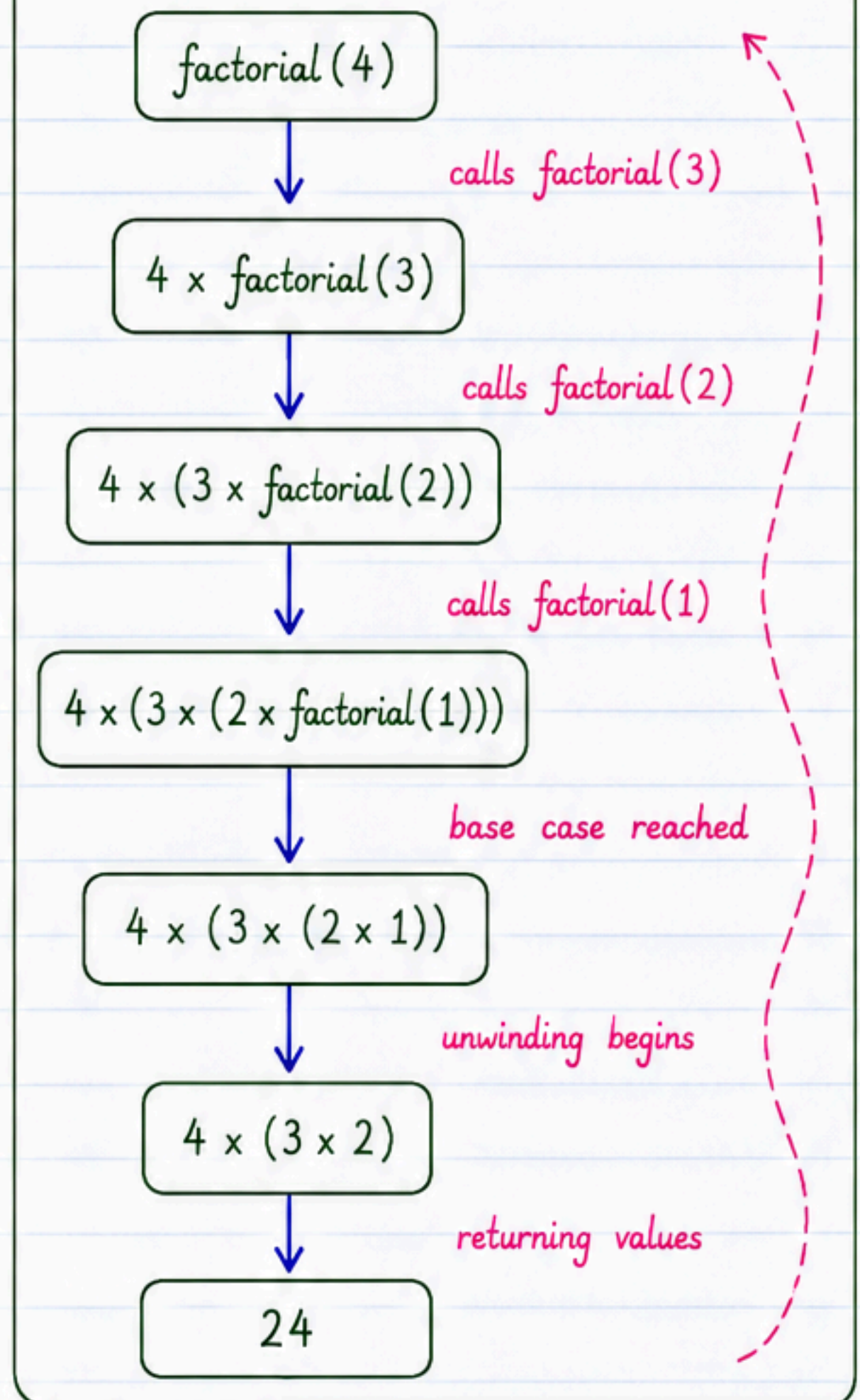
Using the recursive function:

$$\begin{aligned}\text{factorial}(4) &= 4 \times \text{factorial}(3) \\ &= 4 \times (3 \times \text{factorial}(2)) \\ &= 4 \times (3 \times (2 \times \text{factorial}(1))) \\ &= 4 \times (3 \times (2 \times 1)) \\ &= 4 \times (3 \times 2) \\ &= 4 \times 6 \\ &= \underline{\underline{24}}\end{aligned}$$

Break
it down
step by
step!



Recursion Call Flow for `factorial(4)`



- The programmer needs to be extremely careful while working with recursion to ensure that the function doesn't infinitely keep calling itself:

Always have a
base case!



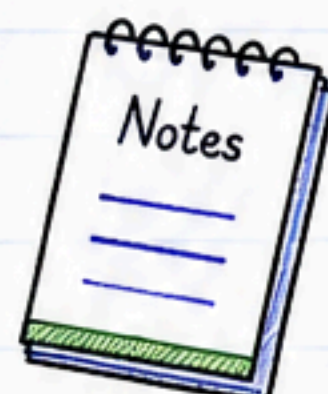
- Recursion is sometimes the most direct way to code an algorithm.



Simple,
Elegant,
Powerful!



Think
Recursively,
Code Smartly!





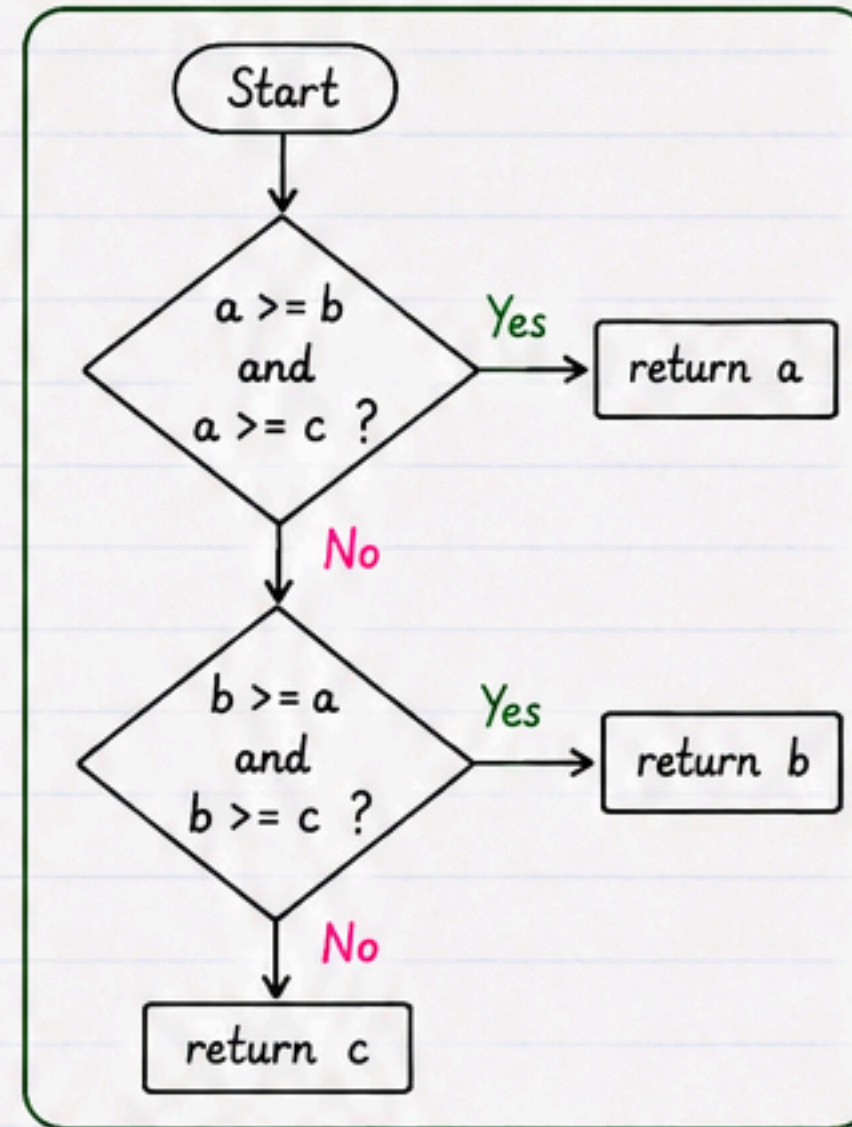
PRACTICE SET



- 1 Write a program using functions to find **greatest** of three numbers.

```
def greatest(a,b,c):  
    if a>=b and a>=c:  
        return a  
    elif b>=a and b>=c:  
        return b  
    else:  
        return c
```

Example usage
x = greatest(10, 25, 15)
print(x) # Output: 25



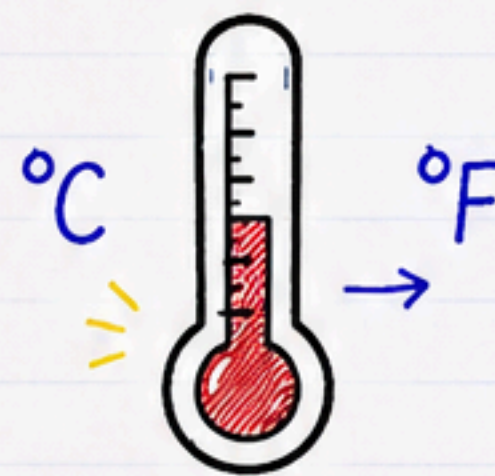
- 2 Write a python program using function to convert **Celsius** to **Fahrenheit**.

```
def celsius_to_fahrenheit(c):  
    f = (c * 9/5) + 32  
    return f
```

Example usage
c = 25
f = celsius_to_fahrenheit(c)
print(f) # Output: 77.0

Formula Used:

$$\text{Fahrenheit (F)} = (\text{Celsius (C)} \times \frac{9}{5}) + 32$$



- 3 How do you prevent a python print() function to print a **new line** at the end. Use the **end** parameter in print().

```
print("Hello", end=" ")  
print("World")  
# Output: Hello World
```

By default, print() adds a \n (new line) at the end. Use end="" to change it.



- 4 Write a recursive function to calculate the **sum** of first **n** natural numbers.

```
def sum_n(n):  
    if n == 0:  
        return 0  
    else:  
        return n + sum_n(n-1)
```

Example usage
n = 5
result = sum_n(n)
print(result)
Output: 15

How it works (for n = 5):

$$\begin{aligned} \text{sum_n}(5) &= 5 + \text{sum_n}(4) \\ &= 5 + (4 + \text{sum_n}(3)) \\ &= 5 + (4 + (3 + \text{sum_n}(2))) \\ &= 5 + (4 + (3 + (2 + \text{sum_n}(1)))) \\ &= 5 + (4 + (3 + (2 + (1 + \text{sum_n}(0)))) \\ &= 5 + (4 + (3 + (2 + (1 + 0)))) \\ &= \underline{\underline{15}} \end{aligned}$$



- 5 Write a python function to print first n lines of the following pattern.

```
def print_pattern(n):  
    for i in range(n, 0, -1):  
        print("*" * i)
```

```
# Example usage  
n = 3  
print_pattern(n)
```

For $n = 3$, Output:

```
***  
**  
*
```

Each line has one star less than the above line!



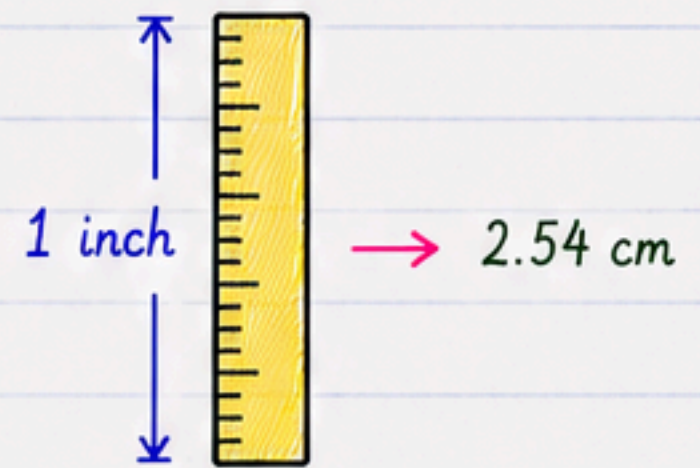
- 6 Write a python function which converts inches to cms.

```
def inches_to_cms(inches):  
    cms = inches * 2.54  
    return cms
```

```
# Example usage  
i = 10  
cms = inches_to_cms(i)  
print(cms)    # Output: 25.4
```

Formula Used:

1 inch = 2.54 cm
 $\text{cms} = \text{inches} \times 2.54$



Length converted easily!



- 7 Write a python function to remove a given word from a list and strip it at the same time.

```
def remove_word(lst, word):  
    new_list = []  
    for item in lst:  
        item = item.strip() # strip spaces  
        if item.lower() != word.lower():  
            new_list.append(item)  
    return new_list
```

```
# Example usage  
lst = [" Hello ", "World ",  
        " python ", " Code "]  
word = "world"  
result = remove_word(lst, word)  
print(result)  
# Output: ['Hello', 'python', 'Code']
```



Removes the word (case-insensitive) and strips extra spaces!

- 8 Write a python function to print multiplication table of a given number.

```
def multiplication_table(n):  
    for i in range(1, 11):  
        print(f"{n} x {i} = {n*i}")
```

```
# Example usage  
n = 5  
multiplication_table(n)
```

Example Output (for $n = 5$):

```
5 x 1 = 5  
5 x 2 = 10  
5 x 3 = 15  
5 x 4 = 20  
5 x 5 = 25  
5 x 6 = 30  
5 x 7 = 35  
5 x 8 = 40  
5 x 9 = 45  
5 x 10 = 50
```



Multiplication made simple!

